WORKSHEET **P9** **Pertemuan 9 · DOM, Event, dan Interaktivitas**

Memasang Data ke Halaman dan Menanggapi Pengguna

Pengembangan Aplikasi Berbasis Web · SIF302 · Semester Gasal 2026/2027

NAMA		**NIM**	
KELAS		**TANGGAL**	

Dikerjakan sendiri, bukan berpasangan. Setiap mahasiswa mengerjakan satu halaman profil miliknya sendiri. Anda bebas memilih isi halaman, tetapi harus bisa menjelaskan setiap baris kode yang Anda tulis.

Capaian pembelajaran	Yang dinilai hari ini
Mahasiswa mampu memilih elemen halaman, menyusun elemen baru dari data, dan menanggapi tindakan pengguna dengan satu pendengar di induk, sehingga halaman berubah tanpa dimuat ulang.	1. Halaman berjalan tanpa galat dan tidak ada pemilih yang menghasilkan null. 2. Daftar proyek terbentuk dari data P8, bukan dari HTML yang ditulis tangan. 3. Wadah dikosongkan lebih dulu setiap kali daftar digambar ulang. 4. Satu pendengar di induk melayani semua tombol filter. 5. Form tidak memuat ulang halaman dan pesan galat muncul per kolom.

Aturan pengerjaan

Hal	Ketetapan
Bentuk kerja	Individu. Satu mahasiswa, satu halaman, satu topik.
Perkakas	VS Code + peramban Chrome atau Firefox. Chrome DevTools dipakai terus-menerus mulai lembar A.
Berkas awal	Hasil pekerjaan Anda sendiri dari Pertemuan 8 — profil.html, seluruh berkas CSS, dan js/app.js — disalin ke folder worksheet-p9/. Tidak ada berkas contoh, tidak ada kerangka yang disediakan.
Waktu	90 menit di kelas. Bila belum selesai, dilanjutkan di luar kelas paling lambat pukul 23.59 pada hari yang sama.
Penyerahan	Seluruh isi worksheet-p9/ diunggah ke repositori GitHub Anda sendiri, di dalam folder semester ~/pabw/. Periksa halaman repositori Anda, bukan hanya folder di laptop.

Anggaran waktu 90 menit

Menit	Lembar	Yang Anda kerjakan	Kalau waktu habis, berhenti di sini
0–18	A	Salin hasil Pertemuan 8 ke worksheet-p9/, buat js/dom.js, catat daftar elemen yang akan diisi	Berkas terhubung dan daftar elemen terisi
18–40	B	Render daftar proyek: map, createElement, textContent, append	Daftar proyek tampil di halaman dari data, bukan dari HTML
40–62	C	Tombol filter kategori dengan satu pendengar di induknya	Klik filter mengubah isi daftar
62–85	D	Form P3 diberi submit + preventDefault, validasi per kolom, pesan galat	Halaman tidak dimuat ulang; pesan galat muncul dan hilang tepat waktu
85–110	E	Tiga kasus sulit: null dari pemilih, pendengar ganda, isi daftar kosong	Tiga kasus ditangani + tangkapan layar
110–150	F	Periksa satu per satu, satu baris untuk diingat, tiket keluar, penilaian mandiri, dan bagian README	Lembar F terisi dan tersimpan

Selalu kerjakan berurutan. Setiap lembar mengandaikan lembar sebelumnya sudah selesai. Kalau

Anda melompat, gejalanya akan sulit dilacak karena sebabnya ada di lembar yang belum Anda kerjakan. Kolom paling kanan bukan batas waktu — hanya penunjuk laju.

Lembar A — Memilih elemen di halaman

PILIH DULU ELEMENNYA, BARU SENTUH ISINYA

Sampai Pertemuan 8 data Anda sudah tersusun, tetapi hanya terlihat di Console. Hari ini data itu dipasang ke halaman. Langkah pertama bukan menulis, melainkan mencari: pastikan elemen yang akan Anda isi benar-benar ada dan benar-benar itu.

A.1 Sediakan dulu wadahnya di profil.html

Sebelum mencari elemen, pastikan elemennya ada. Buka profil.html, lalu ganti daftar proyek yang masih ditulis tangan dengan satu wadah kosong, dan tambahkan baris tombol filter beserta satu paragraf pesan. Isi daftar tidak lagi Anda tulis di HTML — JavaScript yang mengisinya mulai lembar B.

```
<section>
  <h2>Proyek</h2>

  <div id="filter">
    <button data-kategori="semua" class="aktif">Semua</button>
    <button data-kategori="web">Web</button>
    <button data-kategori="data">Data</button>
  </div>

  <ul id="daftar"></ul>
  <p id="pesan-kosong" hidden>Tidak ada proyek pada kategori itu.</p>
</section>
```

Baris yang ditambahkan	Untuk apa	Dipakai di lembar
<ul id="daftar">	Wadah tempat kartu proyek dipasang; kosong sejak awal	B (mengisi), C (menyaring), D (mengosongkan)
dan tiga <button>	Induk tempat satu pendengar dipasang; setiap tombol membawa data-kategori	C (event delegation dan penanda tombol aktif)
data-kategori pada setiap tombol dan pada setiap proyek di app.js	Nilai keduanya dicocokkan saat menyaring; beda satu huruf tidak menghasilkan galat, hanya daftar kosong	C (menyaring data)
<p id="pesan-kosong" hidden>	Pesan yang muncul saat penyaringan tidak menghasilkan isi	D (keadaan kosong di dalam render)

Nilai kategori pada tombol harus sama huruf per huruf dengan label kategori pada setiap proyek di berkas app.js Pertemuan 8. Keduanya sudah Anda tetapkan di lembar D pertemuan itu, dan di situlah pasangan nilainya harus cocok.

A.2 Membaca halaman sebagai pohon dan mencari elemennya

Buka halaman Anda, klik kanan pada satu bagian, lalu pilih Inspect. Panel Elements menampilkan halaman sebagai pohon bersarang. Bentuk yang terlihat di situ adalah bentuk yang dipegang JavaScript — bukan berkas HTML yang Anda tulis. Sebelum menulis pemilih, pastikan nama id atau kelasnya ada di panel itu.

Yang dipakai	Hasilnya	Kapan dipakai
querySelector("#daftar")	Satu elemen pertama yang cocok, atau null bila tidak ada	Mengisi satu tempat tertentu di halaman
querySelectorAll(".filter")	Seluruh elemen yang cocok, sebagai NodeList	Menangani banyak tombol atau banyak kartu
elemen.closest("button")	Induk terdekat yang cocok, mulai dari elemen itu sendiri	Mencari tombol dari elemen yang benar-benar diklik

Yang dipakai	Hasilnya	Kapan dipakai
Array.from(hasil)	NodeList diubah menjadi array biasa	Sebelum memakai map atau filter pada hasil pencarian

UJI PEMILIH DI CONSOLE SEBELUM MENULISNYA DI BERKAS

```
document.querySelector("#daftar");            // elemen, atau null
document.querySelectorAll("#filter button");   // NodeList

// NodeList bukan array: map dan filter tidak ada di sana
Array.from(document.querySelectorAll("#filter button")).map((t) => t.textContent);
```

A.3 Daftar elemen yang akan saya isi

Isi tabel ini sebelum menulis kode. Nama id atau kelas yang Anda catat di sini harus sama huruf per huruf dengan yang ada di profil.html — di situlah sumber galat null yang paling sering terjadi.

Bagian halaman	Pemilih yang saya pakai	Diisi apa	Nama variabel
Daftar proyek			
Baris tombol filter			
Pesan daftar kosong			
Form dan tiap kolomnya			

A.4 Membuat berkas dom.js dan menyambungkannya

Buat berkas baru bernama js/dom.js, lalu tambahkan satu baris <script> lagi sebelum </body>. Berkas app.js dari Pertemuan 8 tidak diubah kecuali kata export pada dua namanya — tanpa itu dom.js tidak bisa memakai datanya. Berkas dom.js memakai data itu untuk mengisi halaman.

```
  <script type="module" src="js/app.js"></script>
  <script type="module" src="js/dom.js"></script>
</body>
```

Bagian berkas	Untuk apa	Akibat bila salah
js/dom.js dipisah dari app.js	app.js menyimpan data dan fungsi murni; dom.js menyentuh halaman. Batas ini yang dipecah lagi di Pertemuan 11	Semua ditulis di satu berkas: sulit dicari ketika halaman tidak berubah
Urutan dua baris <script>	app.js lebih dulu, karena dom.js memakai data darinya	dom.js berjalan tanpa datanya dan langsung berhenti
type="module" pada keduanya	Mengizinkan kata export dan import dipakai; tanpa itu peramban berhenti pada kata export pertama	Console: Cannot use import statement outside a module
export pada data di app.js, import di dom.js	Dua berkas modul tidak berbagi lingkup; hanya nama yang diekspor bisa dilihat berkas lain. Karena itu berbagi data wajib lewat export dan import	ReferenceError: daftarProyek is not defined, dan seluruh lembar B berhenti

Sudah dikerjakan	Tanda
Folder worksheet-p9/ berisi profil.html, seluruh CSS, dan js/app.js dari Pertemuan 8	
profil.html sudah punya ul id="daftar", div id="filter" bertombol data-kategori, dan p id="pesan-kosong" dengan atribut hidden	
Berkas js/dom.js sudah dibuat dan disambungkan sebelum </body>	
Seluruh pemilih di tabel A.3 sudah diuji di Console dan tidak ada yang null	

Sudah dikerjakan	Tanda
Halaman dibuka lewat server lokal; Console tidak menampilkan pesan 404	
Panel Elements sudah dipakai sekali untuk memastikan nama id dan kelasnya	

Kalau pemilih Anda menghasilkan null, jangan menebak. Tulis pemilihnya di Console, lalu bandingkan hasilnya dengan panel Elements. Sembilan dari sepuluh galat null berakhir pada satu huruf yang berbeda.

SIMPAN KE GIT — SETELAH LEMBAR A

```
git add .
git commit -m "pesan Anda"
git push

# pesan commit yang disarankan:
# P9 — pilih elemennya dulu
```

Lembar A sudah selesai? Simpan hasilnya sekarang, jangan menunggu kelas berakhir. Simpanan ini tidak memerlukan langkah tambahan apa pun — hanya memastikan pekerjaan Anda tidak bergantung pada satu berkas di satu komputer.

A — simpan perubahan

Cara memastikan: buka Console dan panggil setiap variabel elemen Anda. Tidak ada satu pun yang bernilai null.

Lembar B — Menyusun elemen dari data

SATU DATA, SATU WUJUD DI HALAMAN

Sekarang bentuk nyata pekerjaan hari ini: daftar proyek yang selama ini hanya terlihat sebagai tabel di Console harus tampil di halaman. Yang menulis kartunya bukan Anda, melainkan JavaScript — satu kartu untuk satu isi array, sebanyak isinya.

B.1 createElement, textContent, lalu append

Ada dua cara mengisi halaman, dan pilihannya bukan selera. Menempelkan teks HTML dengan innerHTML lebih pendek, tetapi peramban membacanya sebagai HTML — termasuk apa pun yang diketikkan pengguna. Menyusun elemen satu per satu lebih panjang beberapa baris, dan halaman Anda tidak bisa disusupi.

Yang dipakai	Perannya	Akibat bila tertukar
createElement("li")	Membuat elemen baru yang belum berada di halaman	Elemen yang dibuat tidak muncul sampai dimasukkan ke wadah
textContent = nilai	Mengisi teks; apa pun isinya dianggap teks	Memakai innerHTML: teks dari pengguna dijalankan sebagai HTML (XSS)
append(anak)	Memasukkan elemen sebagai anak terakhir wadah	append(" ") menyisipkan teks, bukan elemen
className = "kartu"	Memberi kelas supaya tampil seperti elemen lain	Tanpa kelas, elemennya ada tetapi tidak berformat
DocumentFragment	Wadah sementara di luar halaman; halaman digambar sekali	Tanpa itu, halaman digambar ulang setiap kali satu kartu ditambahkan

SATU IS ARRAY, SATU KARTU

Dua baris pertama dikerjakan lebih dulu, dan urutannya menentukan. Baris import mengambil data dari app.js — tanpa itu daftarProyek tidak dikenal di berkas ini, dan Console berhenti dengan ReferenceError sebelum satu kartu pun terbentuk.

```
import { daftarProyek } from "./app.js";

const wadah = document.querySelector("#daftar");
const kosong = document.querySelector("#pesan-kosong");

function buatKartu(proyek) {
  const li = document.createElement("li");
  li.className = "kartu";
  li.textContent = proyek.judul;      // teks, bukan HTML
  return li;
}

daftarProyek.forEach((proyek) => wadah.append(buatKartu(proyek)));
```

B.2 Wadah harus dikosongkan sebelum diisi ulang

Bila tombol filter ditekan dua kali dan daftarnya bertambah panjang setiap kali, penyebabnya satu: wadahnya tidak pernah dibersihkan. Baris pengosongan itu wajib ada di baris pertama fungsi render, sebelum apa pun ditambahkan.

Baris	Yang dilakukan	Catatan
wadah.textContent = ""	Membuang seluruh isi wadah beserta anaknya	Paling ringkas; cukup untuk pekerjaan hari ini

Baris	Yang dilakukan	Catatan
wadah.replaceChildren()	Membuang seluruh anak tanpa menyentuh apa pun yang lain	Dipilih bila wadah juga memuat elemen yang harus bertahan
wadah.innerHTML = ""	Bekerja, tetapi memaksa peramban mengurai ulang HTML	Tidak dipakai di pertemuan ini

B.3 Bandingkan hasil kerja Anda

Buka Console dan bandingkan tiga hal sebelum melangkah ke lembar C. Ketiganya bisa Anda periksa sendiri tanpa bertanya.

Yang diperiksa	Hasil yang benar	Hasil yang saya dapat
Jumlah kartu di halaman	Sama dengan panjang daftarProyek	
Satu kartu paling atas	Judulnya sama dengan data pertama	
Teks di dalam kartu	Tampil sebagai teks, bukan tag yang terurai	

Bila daftar tampak benar tetapi Console menampilkan pesan merah, pekerjaan Anda belum selesai. Galat yang tersisa berarti ada bagian yang berhenti sebelum selesai — dan bagian itu akan terlihat di lembar C.

SIMPAN KE GIT — SETELAH LEMBAR B

```
git add .
git commit -m "pesan Anda"
git push

# pesan commit yang disarankan:
# P9 — daftar tampil dari data
```

Lembar B sudah selesai? Simpan hasilnya sekarang, jangan menunggu kelas berakhir. Simpanan ini tidak memerlukan langkah tambahan apa pun — hanya memastikan pekerjaan Anda tidak bergantung pada satu berkas di satu komputer.

B — simpan perubahan

Cara memastikan: muat ulang halaman tanpa menyentuh Console. Daftar proyek langsung tampil, dan tidak ada kartu yang berlipat.

Lembar C — Satu pendengar untuk semua tombol

SATU PENDENGAR DI INDUK, SEMUA ANAK TERLAYANI

Klik pertama yang harus Anda tangani adalah tombol filter. Cara yang paling mudah dipahami pemula — memasang pendengar pada setiap tombol — punya satu kelemahan: kartu dan tombol yang dibuat kemudian tidak punya pendengar. Karena itu pendengarnya dipasang di induknya, sekali saja.

C.1 addEventListener dan event delegation

Pendengar di induk menerima seluruh klik yang terjadi di dalam dirinya, termasuk dari anak yang belum ada saat pendengar itu dipasang. Yang perlu Anda lakukan hanya memeriksa elemen mana yang benar-benar diklik.

```
const barisFilter = document.querySelector("#filter");

barisFilter.addEventListener("click", (event) => {
  const tombol = event.target.closest("button");
  if (!tombol) return;              // klik di luar tombol, abaikan

  const kategori = tombol.dataset.kategori;
  const terpilih = daftarProyek.filter(
    (proyek) => kategori === "semua" || proyek.kategori === kategori
  );

  render(terpilih);
});
```

Istilah	Artinya	Catatan
event.target	Elemen yang benar-benar diklik	Bisa jadi anak dari tombol, bukan tombolnya sendiri
event.currentTarget	Elemen tempat pendengar dipasang	Di dalam arrow function, nilainya tetap induknya
closest("button")	Menaikkan pandangan ke tombol terdekat	Bila bukan dari tombol, hasilnya null
dataset.kategori	Membaca atribut data-kategori	Cara memberi keterangan pada tombol tanpa menulis delapan if

C.2 Tombol yang sedang aktif

Pengguna perlu tahu filter mana yang sedang berlaku. Tandai tombol aktif dengan satu kelas, bukan dengan warna yang ditulis langsung di JavaScript. Tulisannya tetap di CSS; JavaScript hanya menambahkan dan melepas kelasnya.

Aturan CSS-nya ditambahkan sendiri ke komponen.css. Tanpa itu tombolnya tetap bekerja, tetapi penanda aktifnya tidak terlihat. Nama kelas .aktif harus sama huruf per huruf dengan yang ditulis JavaScript di bawah.

```
.aktif {
  outline: 2px solid currentColor;
  outline-offset: 2px;
}
function tandaiTombolAktif(tombolAktif) {
  document.querySelectorAll("#filter button").forEach((tombol) => {
    tombol.classList.toggle("aktif", tombol === tombolAktif);
  });
}
```

Keadaan	Yang harus terjadi	Hasil yang saya dapat

Keadaan	Yang harus terjadi	Hasil yang saya dapat
Halaman baru dibuka	Semua proyek tampil, tombol "semua" bertanda aktif	
Klik satu kategori	Hanya proyek kategori itu yang tampil	
Klik kategori kosong	Wadah kosong dan pesannya muncul, bukan halaman kosong	
Klik dua kali cepat	Jumlah kartu tidak berlipat	

Bila kartu berlipat setiap kali tombol ditekan, jangan menambah kode baru. Periksa dulu apakah baris pengosongan wadah masih ada di fungsi render Anda.

SIMPAN KE GIT — SETELAH LEMBAR C

```
git add .
git commit -m "pesan Anda"
git push

# pesan commit yang disarankan:
# P9 — filter bekerja dengan satu pendengar
```

Lembar C sudah selesai? Simpan hasilnya sekarang, jangan menunggu kelas berakhir. Simpanan ini tidak memerlukan langkah tambahan apa pun — hanya memastikan pekerjaan Anda tidak bergantung pada satu berkas di satu komputer.

C — simpan perubahan

Cara memastikan: tekan tiga tombol filter bergantian. Isi daftar berubah, jumlah kartu tidak berlipat, dan Console tetap bersih.

Lembar D — Pola render dan validasi form

TAMPILAN ADALAH FUNGSI DARI DATA

Sampai lembar C, tampilan Anda tumbuh dari beberapa tempat: satu fungsi menambah kartu, satu bagian menyembunyikan pesan, satu baris lagi menandai tombol aktif. Pada lembar ini seluruhnya dikumpulkan ke satu fungsi render(data). Setelah itu tidak ada tempat kedua yang perlu Anda perbarui.

D.1 Satu fungsi render untuk satu bagian halaman

Fungsi render punya tiga pekerjaan tetap, dan urutannya tidak boleh ditukar: kosongkan wadahnya, periksa apakah datanya kosong, lalu isi ulang. Ukuran keberhasilannya sederhana — setiap perubahan data diikuti tepat satu pemanggilan render.

```
function render(daftar) {
  wadah.textContent = "";              // 1. kosongkan lebih dulu

  if (daftar.length === 0) {           // 2. periksa keadaan kosong
    kosong.hidden = false;
    return;
  }
  kosong.hidden = true;

  daftar.forEach((proyek) => wadah.append(buatKartu(proyek)));   // 3. isi ulang
}
```

Aturan	Alasannya	Yang terjadi bila dilanggar
Satu fungsi render per bagian halaman	Satu tempat yang harus benar; mudah diperiksa	Tampilan berbeda dari data karena ada bagian yang terlewat diperbarui
Wadah dikosongkan di baris pertama	Isi lama dan isi baru tidak boleh bertumpuk	Kartu berlipat setiap kali render dipanggil
Keadaan kosong ditangani di dalam render	Pengguna tidak boleh melihat halaman yang diam	Penyaringan yang tidak menghasilkan apa pun terlihat seperti kerusakan
Pendengar dipasang di luar render	Render boleh dipanggil berkali-kali tanpa menambah pendengar	Satu klik dijalankan berkali-kali, Console mencetak berulang

D.2 Validasi form yang menjawab sebelum dikirim

Form sudah ada di halaman Anda sejak Pertemuan 3. Sekarang ia diberi tiga hal: preventDefault supaya halaman tidak dimuat ulang, validasi yang berjalan saat pengguna mengetik, dan pesan galat per kolom. Pesannya menyebut cara memperbaiki, bukan menyalahkan.

Yang dipakai	Perannya	Kapan dijalankan
event.preventDefault()	Menghentikan pengiriman bawaan peramban	Di baris pertama penangan submit
input.value.trim()	Membaca isi kolom tanpa spasi di tepinya	Setiap kali memeriksa kolom yang boleh kosong
setAttribute("aria-invalid", "true")	Menandai kolom bermasalah bagi pembaca layar	Saat kolom itu dinyatakan tidak sah
tombol.disabled = !sah	Menahan pengiriman sampai seluruh kolom layak	Setiap kali hasil pemeriksaan berubah
kolom.focus()	Memindahkan pandangan ke kolom yang perlu diperbaiki	Sesudah pemeriksaan selesai dan ada kolom bermasalah

D.3 Periksa hasil kerja Anda

Yang diperiksa	Hasil yang benar	Hasil yang saya dapat
Kirim form kosong	Halaman tidak dimuat ulang; pesan galat muncul	
Perbaiki satu kolom	Pesannya hilang begitu isinya layak	
Isi hanya spasi	Masih dinyatakan tidak sah	
Tombol kirim	Menunggu sampai seluruh kolom layak	

Form adalah satu-satunya bagian halaman yang bisa menghapus seluruh hasil kerja Anda dalam satu klik. Pastikan halaman tidak dimuat ulang sebelum menambah validasi apa pun.

SIMPAN KE GIT — SETELAH LEMBAR D

```
git add .
git commit -m "pesan Anda"
git push

# pesan commit yang disarankan:
# P9 — render dan validasi
```

Lembar D sudah selesai? Simpan hasilnya sekarang, jangan menunggu kelas berakhir. Simpanan ini tidak memerlukan langkah tambahan apa pun — hanya memastikan pekerjaan Anda tidak bergantung pada satu berkas di satu komputer.

D — simpan perubahan

Cara memastikan: kirim form kosong, lalu perbaiki satu kolom. Halaman tidak dimuat ulang dan pesan galatnya hilang tepat setelah kolomnya benar.

Lembar E — Membaca gejala, bukan menebaknya

YANG PALING SERING GAGAL BUKAN KODENYA, TAPI URUTANNYA

Pertemuan ini punya satu jenis kegagalan yang khas: halaman yang tampak tidak bereaksi, padahal Console bersih. Tidak ada pesan merah yang bisa Anda baca — yang ada hanya halaman yang diam. Lembar ini melatih cara membaca gejala semacam itu.

E.1 Memeriksa pendengar dengan DevTools

Pendengar yang terpasang dua kali tidak menimbulkan galat. Yang terlihat hanya satu tindakan yang berjalan berkali-kali. Panel Event Listeners menunjukkan pendengar mana yang menempel pada elemen mana.

Yang dilakukan	Di mana	Yang Anda lihat
Klik kanan → Inspect	Panel Elements	Elemen itu disorot di pohon halaman
Tab Event Listeners	Panel di sebelah Elements	Daftar peristiwa yang menempel pada elemen itu
monitorEvents(el)	Console	Setiap peristiwa pada elemen itu tercetak saat terjadi
Hitung baris Console	Console, saat tombol ditekan	Satu baris bila benar; lebih dari satu bila pendengarnya ganda

E.2 Halaman diam — empat sebab yang paling sering

Urutan pemeriksaan di bawah ini disusun dari yang paling sering terjadi. Kerjakan berurutan; jangan melompat ke sebab yang paling rumit.

Gejala	Sebab yang paling sering	Yang dilakukan
Halaman tampak normal, tidak ada apa pun yang berubah	Skrip berjalan sebelum elemennya ada, sehingga pemilihnya null	Periksa urutan <script>: app.js lalu dom.js, keduanya sebelum </body>
Daftar tampil, tetapi filter tidak mengubah apa pun	Pendengar dipasang pada tombol yang belum ada, atau pada elemen yang salah	Pindahkan pendengar ke induknya, lalu periksa event.target.closest("button")
Satu klik membuat daftar berlipat	addEventListener dipanggil di dalam fungsi render	Pindahkan pendengar ke luar fungsi render, dipasang sekali saja
Daftar kosong tanpa penjelasan	Penyaringan tidak menghasilkan isi, dan keadaan kosong tidak ditangani	Tampilkan pesan di dalam render ketika daftar.length === 0
Console: Cannot read properties of null	querySelector tidak menemukan elemennya	Bandingkan penulisan id atau kelas dengan yang ada di panel Elements

E.3 Tiga kasus sulit dan cara menanganinya

Yang muncul	Artinya	Yang dilakukan
Pemilih menghasilkan null	Nama id atau kelasnya berbeda, atau elemennya belum ada saat skrip berjalan	Uji pemilihnya di Console lebih dulu; periksa urutan <script>
Pendengar ganda setelah render ulang	Pendengar dipasang ulang setiap kali wadahnya diisi ulang	Pasang pendengar sekali di induk, di luar fungsi render
Isi daftar kosong	Penyaringan tidak menghasilkan apa pun, atau datanya belum terbaca	Tampilkan pesan keadaan kosong; periksa panjang array dengan console.log

E.4 Catat kasus yang saya temui

Gejala yang saya lihat	Sebabnya	Baris yang saya ubah

Gejala yang saya lihat	Sebabnya	Baris yang saya ubah

Simpan tiga tangkapan layar: satu saat halaman diam beserta panel Event Listeners-nya, satu setelah diperbaiki, dan satu saat filter bekerja. Ketiganya Anda perlukan di lembar F.

SIMPAN KE GIT — SETELAH LEMBAR E

```
git add .
git commit -m "pesan Anda"
git push

# pesan commit yang disarankan:
# P9 — gejala dibaca, bukan ditebak
```

Lembar E sudah selesai? Simpan hasilnya sekarang, jangan menunggu kelas berakhir. Simpanan ini tidak memerlukan langkah tambahan apa pun — hanya memastikan pekerjaan Anda tidak bergantung pada satu berkas di satu komputer.

E — simpan perubahan

Cara memastikan: halaman bereaksi pada setiap klik, dan tabel E.4 terisi dengan sebab yang benar — bukan dengan "sudah diperbaiki" saja.

TIKET KELUAR

Dikerjakan sendiri, tanpa berdiskusi. Lembar ini mengukur pemahaman Anda, bukan kecepatan Anda menyalin jawaban teman.

1. Sebutkan satu elemen di halaman Anda, pemilih yang Anda pakai untuk mengambilnya, dan nama variabel tempat hasilnya disimpan.

2. Apa bedanya querySelector dan querySelectorAll? Mengapa map tidak langsung bisa dipakai pada hasil querySelectorAll?

3. Anda memasang pendengar di induk, bukan pada setiap tombol. Apa keuntungannya, dan bagian mana dari kode Anda yang membuktikannya?

4. Apa yang terjadi bila baris pengosongan wadah dihapus dari fungsi render? Peragakan, lalu tulis hasilnya.

5. Mengapa isi yang berasal dari pengguna tidak boleh dimasukkan dengan innerHTML? Tulis cara yang Anda pakai sebagai gantinya.

F.1 Periksa satu per satu

Jangan menyerahkan pekerjaan sebelum menjalankan daftar ini. Semua yang diperiksa di bawah bisa Anda lihat sendiri dalam lima menit, dan semuanya benar-benar dinilai.

- ☐ Berkas profil.html, seluruh CSS, js/app.js, dan js/dom.js ada di repositori dan terbuka tanpa galat di Console
- ☐ Pemilih setiap pemilih menghasilkan elemen; tidak ada satu pun yang bernilai null
- ☐ Render dari data daftar proyek terbentuk dari array dengan createElement dan textContent, bukan dari HTML yang ditulis tangan
- ☐ Wadah dikosongkan di baris pertama fungsi render, sehingga kartu tidak pernah berlipat
- ☐ Event delegation satu pendengar di induk melayani semua tombol filter; event.target dan closest dipakai benar
- ☐ Keadaan kosong halaman menampilkan pesan ketika penyaringan tidak menghasilkan isi

☐ Validasi form halaman tidak dimuat ulang, pesan galat muncul per kolom, dan tombol kirim menunggu sampai isinya layak
☐ Deklarasi AI bagian README menyebutkan bagian mana yang dibantu AI dan bagian mana yang Anda kerjakan sendiri
☐ Git ada lebih dari satu commit hari ini, dan pesan commit-nya menjelaskan isinya

F.2 Satu baris untuk diingat

Tulis satu kalimat — bukan untuk dosen, untuk Anda sendiri dua pekan dari sekarang — tentang hal yang hari ini paling sering membuat Anda salah.

F.3 Penilaian mandiri

Isi perkiraan nilai Anda apa adanya sebelum menyerahkan lembar ini. Penilaian yang jujur membantu Anda mengetahui bagian mana yang belum kokoh. Kolom Bukti diisi nama berkas atau baris kode yang bisa dibuka dosen.

☐ Pengungkapan AI README menyebutkan bagian mana yang dibantu AI dan bagian mana yang saya kerjakan sendiri
☐ Keaslian topik dan isi halaman ini milik saya sendiri, tidak menyalin pekerjaan mahasiswa lain

Bagian	Bobot	Nilai saya	Bukti
Pemilihan dan pengisian elemen	20		
Render dari data	25		
Event dan event delegation	25		
Validasi form	20		
Kebersihan kode, deklarasi AI, dan bukti	10		
TOTAL	100		

F.4 Catatan untuk pengampu

Bagian tersulit hari ini:

Yang ingin saya bahas di kelas berikutnya:

Tanda tangan: _____ Tanggal: _____ Jam selesai: _____

SIMPAN KE GIT — SETELAH LEMBAR F

```
git add .
git commit -m "pesan Anda"
git push

# pesan commit yang disarankan:
# P9 — periksa, lalu simpan
```

Lembar F sudah selesai? Simpan hasilnya sekarang, jangan menunggu kelas berakhir. Simpanan ini tidak memerlukan langkah tambahan apa pun — hanya memastikan pekerjaan Anda tidak bergantung pada satu berkas di satu komputer.

F — simpan perubahan

Cara memastikan: buka README di repositori Anda. Bagian Pertemuan 9 sudah memuat deklarasi AI, dan commit terakhir adalah commit terbaru.

Lampiran A — Rubrik penilaian

RUBRIK PENILAIAN total 100

Aspek	Bobot	Nilai 100 — sangat baik	Nilai 65 — memadai	Nilai 35 — kurang
Pemilihan dan pengisian elemen	20%	Pemilih tepat dan tidak ada yang null; isi halaman dimasukkan dengan textContent; kelas dipakai untuk format	Sebagian pemilih masih perlu diperbaiki, atau ada isi yang dimasukkan dengan innerHTML	Elemen belum terpilih dengan benar, atau halaman tidak berubah sama sekali
Render dari data	25%	Daftar terbentuk dari array; wadah dikosongkan lebih dulu; urutan kartu sama dengan urutan datanya	Daftar tampil, tetapi kartu berlipat atau urutannya berbeda dari datanya	Daftar masih ditulis tangan di HTML, atau render menghasilkan undefined
Event dan event delegation	25%	Satu pendengar di induk melayani semua tombol; target dan closest dipakai benar; tidak ada pendengar ganda	Pendengar bekerja tetapi dipasang per tombol, atau ada pendengar yang bertambah setiap render	Klik tidak menghasilkan perubahan, atau satu klik menjalankan beberapa tindakan
Validasi form	20%	Halaman tidak dimuat ulang; pesan galat per kolom menyebut cara memperbaiki; tombol menunggu sampai isinya layak	Validasi berjalan, tetapi pesannya umum atau tombol kirim belum ditahan	Form masih memuat ulang halaman, atau validasi belum ada
Kebersihan kode, deklarasi AI, dan bukti	10%	Penamaan konsisten, batas app.js dan dom.js jelas; deklarasi AI terisi di README; tangkapan layar disertakan	Kode rapi tetapi deklarasi AI belum lengkap, atau bukti belum lengkap	Deklarasi AI kosong, atau pekerjaan tidak tersimpan di repositori

Yang dinilai adalah pekerjaan Anda sendiri. Topik yang berbeda antar mahasiswa membuat penilaian tidak bisa dibandingkan antar pekerjaan — karena itu yang diperiksa adalah apakah setiap bagian bekerja sebagaimana dimaksud, bukan apakah ia sama dengan contoh.

Lampiran B — Rujukan cepat dan kalau macet

REFERENSI CEPAT DOM hafalkan yang ini

Yang dipakai	Perannya	Kapan dipakai
querySelector	Mengambil satu elemen pertama yang cocok	
querySelectorAll	Mengambil seluruh elemen yang cocok sebagai NodeList	
Array.from(NodeList)	Mengubah NodeList menjadi array	
createElement	Membuat elemen baru	
textContent	Mengisi teks dengan aman	
append	Memasukkan elemen sebagai anak terakhir	
replaceChildren()	Mengosongkan wadah dari seluruh anaknya	
classList.toggle	Menambah atau melepas kelas	
addEventListener	Memasang pendengar	
event.target.closest	Mencari elemen induk terdekat yang cocok	

Yang dipakai	Perannya	Kapan dipakai
preventDefault()	Menghentikan perilaku bawaan peramban	
dataset	Membaca atribut data-*	

KALAU MACET cari gejalanya di sini

Gejala	Penyebab yang paling sering	Yang dilakukan
console: Cannot read properties of null	querySelector tidak menemukan elemennya	Bandingkan penulisan id atau kelas dengan panel Elements
Halaman tidak berubah saat tombol ditekan	Pendengar dipasang pada elemen yang salah, atau belum ada saat dipasang	Pasang pendengar di induknya dan periksa event.target
Klik pertama bekerja, klik kedua menggandakan	addEventListener dipanggil di dalam fungsi render	Pindahkan pemasangan pendengar ke luar render
Kartu bertambah terus di daftar	Wadah tidak dikosongkan sebelum diisi ulang	Tambahkan wadah.textContent = "" di baris pertama render
Daftar kosong tanpa keterangan	Penyaringan tidak menghasilkan isi	Tampilkan pesan ketika daftar.length === 0
Isi kartu tampil sebagai tag, bukan teks	Isi dimasukkan dengan innerHTML	Ganti dengan textContent
Form memuat ulang halaman	preventDefault tidak dipanggil pada penangan submit	Panggil event.preventDefault() di baris pertama
Validasi lolos pada kolom berisi spasi	Isi diperiksa tanpa trim()	Periksa dengan kolom.value.trim() !== ""
Console: 404 (Not Found) untuk dom.js	Jalur di atribut src tidak cocok dengan letak berkasnya	Pastikan dom.js berada di dalam folder js/, huruf besar-kecilnya sama
Perubahan pada salinan ikut mengubah data asli	Array disalin dengan = , bukan [...]	Ganti menjadi const salinan = [...daftarProyek]

Halaman yang Anda bangun hari ini adalah bahan ujian CPMK Blok 2 nanti: pemilihan elemen, render dari data, dan event delegation semuanya diuji di sana. Simpan pekerjaan ini rapi di repositori Anda — Anda akan memakainya lagi di Pertemuan 11.

LAMPIRAN — MENYIMPAN PEKERJAAN ANDA KE GIT sekali saja, bukan tiap pertemuan

Lampiran ini bukan pengantar Git. Panduan Git dan GitHub yang sudah Anda pakai sejak Pertemuan 3 masih berlaku seluruhnya; bagian ini hanya mengulang tiga perintah yang paling sering dipakai, karena mulai hari ini ada berkas baru yang harus ikut disimpan.

Tiga perintah yang akan Anda pakai terus

Perintah	Artinya	Kapan dijalankan
git add .	Menandai seluruh perubahan di folder ini untuk disimpan	Setiap kali selesai satu lembar
git commit -m "pesan"	Menyimpan perubahan itu sebagai satu riwayat	Langsung setelah git add
git push	Mengirim riwayat itu ke GitHub	Setelah commit, supaya dosen bisa melihatnya

Titik pada git add . Titik itu berarti "seluruh berkas di folder ini". Tanpa titik, Git tidak tahu apa yang harus disimpan. Hal yang sama berlaku untuk berkas dom.js yang baru Anda buat — ia ikut tersimpan

karena berada di dalam folder semester.

Berapa kali harus menyimpan?

Tidak ada angka yang benar, tetapi patokannya begini: bila Anda sudah menyelesaikan satu hal yang berarti dan bisa Anda jelaskan, simpanlah. Pada pertemuan ini minimal dua kali: setelah daftar tampil dari data, dan setelah filter serta validasi selesai.

Kalau ini terjadi	Yang Anda lakukan
Sudah menulis banyak, belum pernah commit	Simpan sekarang juga
Salah commit, belum push	Perbaiki lalu commit lagi — jangan hapus repositori
Sudah push, lalu menemukan kekeliruan	Perbaiki berkasnya dan commit lagi; riwayat lama tidak hilang dan itu wajar

Yang dinilai bukan jumlah commit. Menyimpan sering membuat pekerjaan Anda tidak bergantung pada satu berkas di satu komputer — dan pada pertemuan ujian, repositori itulah yang menentukan apakah pekerjaan Anda bisa dinilai.